Notes for C++

Section 6 : Functions

Definition

A function is a named self-contained section of a program that performs a logical task. A function can be thought of as a "mini-program" with variables, statements, etc. Some functions have been created by other programmers (e.g. `sqrt`) and these functions are available to us from the libraries (e.g. `cmath`). This section is about functions that we write for ourselves. These are known as ***user-defined functions.***

Example: A simple function

```
#include <iostream>
using namespace std;

void prompt();                                  /* function prototype */

int x;

int main() {
  prompt();                                     /* function call */
  cin >> x;
  cout << "You entered " << x;
}

void prompt() {                                 /* function definition */
  cout << "Please type in a number\n";
  cout << "The number must be from 0 to 10\n";
  cout << "Press Enter after typing the number\n";
}
```

Interesting things to notice in this program:

- the function prototype and the first line of the function definition are the same
- when `main` ***calls*** the function, `main` stops temporarily, while the statements inside the function definition are executed – in this example there are 3 statements inside the function
- when the function ends (completes all its instructions) then main continues
- we use the word `void` – this will be discussed further on in this section

Program layout

- **function prototype –** void before the name, empty parentheses after the name. The prototypes for all functions appear at the top of the program.
- **function call –** the name and empty parentheses. Appears somewhere in main, could appear several times, could appear inside another function.
- **function definition –** the instructions to carry out the task. Appears after main and each function has its own definition. This is the "self-contained section" of code.

Example: Reading two test results – version 1 (no parameters)

```
#include <iostream>
using namespace std;

void readmark();

int test1, test2, maxmark, temp, result;
/* note - test 1 is out of 30 and test 2 is out of 70 */

int main() {

// section 1: get the mark for test 1
  maxmark = 30;
  readmark();
  test1 = temp;

// section 2: get the mark for test 2
  maxmark = 70;
  readmark();
  test2 = temp;

// section 3: calculate the final result
  result = test1 + test2;
  cout << "Final result is " << result;
}

void readmark() {
  cout << "Enter the mark for the test ";
  cin >> temp;
  while ((temp < 0) || (temp > maxmark)) {
    cout << "ERROR: mark must be from 0 to " << maxmark << endl;
    cout << "Enter the mark again ";
    cin >> temp;
  }
}
```

Interesting things to notice in this program:

- the function is called more than once
- when we want to use the function, we have to remember to set the correct maximum mark before calling the function – we use `maxmark` for this.
- after using the function, we have to remember that the test result is stored in `temp` – we have to copy this to a different variable for each test result.
- is there a better way of passing data and results to and from functions? Yes, there is and we will cover this in the next few examples.

Example: A function with one parameter

```
#include <iostream>
using namespace std;

void prompt(int maxi);                  /* function prototype */
```

```
int a, b;

int main() {
  prompt(33);                          /* function call */
  cin >> a;
  prompt(67);                          /* function call */
  cin >> b;
  cout << "You entered " << a << " and " << b;
}

void prompt(int maxi) {                /* function definition */
  cout << "Please type in a number\n";
  cout << "The number must be from 0 to " << maxi << endl;
  cout << "Press Enter after typing the number\n";
}
```

When this program runs, the display on the screen will look like this:
Note that in this example we assume that the person using the program types the numbers 10 and 55.

```
Please type in a number
The number must be from 0 to 33
Press Enter after typing the number
10
Please type in a number
The number must be from 0 to 67
Press Enter after typing the number
55
You entered 10 and 55
```

Interesting things to notice in this program:

- the function has a ***parameter*** (or ***argument***) called `maxi`
- when the program ***calls*** the function, it ***passes*** a value to `maxi` in the function's area of memory
- the value passed to an `int` or `float` parameter is a **copy** of the value the function is called on
- the data type of the value the function is called on matches the data type in the prototype
- the function uses the parameter as a normal variable (an integer variable in this example)
- parameters can only be accessed by the function they belong to e.g. if you use `max` in main you would get an error
- a function can be called on an actual value, or a variable, or a calculation, for example:
  `prompt(67);` (67 passed to `maxi`)
  `prompt(a);` (10 passed to the function in the example above)
  `prompt((a + 50) / 2);` (30 passed to `max` in the example above)

Example: A function with two parameters

```
#include <iostream>
using namespace std;

void prompt(int mini, int maxi);       /* function prototype */

int a, b;

int main() {
```

```
  prompt(12, 33);                    /* function call */
  cin >> a;
  prompt(85, 99);                    /* function call */
  cin >> b;
  cout << "You entered " << a << " and " << b;
}

void prompt(int mini, int maxi) {      /* function definition */
  cout << "Please type in a number\n";
  cout << "The number must be from " << mini;
  cout << " to " << maxi << endl;
  cout << "Press Enter after typing the number\n";
}
```

Interesting things to notice in this program:

- the function has ***two*** parameters – note the syntax: `int mini, int maxi`
- when the program ***calls*** the function, it ***passes*** two values to the function
- the values are loaded into the parameters **in the order given** (e.g. `mini` gets 12 and not 33)
- the number, order and data types in the function call match those in the prototype
- the function uses the parameters as normal variables
- the parameters are unknown inside main (use of `mini` or `maxi` in main would be an error)

Example: Reading two test results – version 2 (with a parameter)

Look back at the example program to read test results – version 1. We had to use a global variable to send data to the function. Let us write that program again using parameters.

```
#include <iostream>
using namespace std;

void readmark(int maxmark);

int test1, test2, temp, result;

/* note - test 1 is out of 30 and test 2 is out of 70 */

int main() {
  readmark(30);
  test1 = temp;
  readmark(70);
  test2 = temp;
  result = test1 + test2;
  cout << "Final result is " << result;
}

void readmark(int maxmark) {
  cout << "Enter the mark for the test ";
  cin >> temp;
  while ((temp < 0) || (temp > maxmark)) {
    cout << "ERROR: mark must be from 0 to " << maxmark << endl;
    cout << "Enter the mark again ";
    cin >> temp;
  }
```

```
}
```

Interesting things to notice in this program:

- `maxmark` has changed from being a global variable to a parameter
- we no longer have to load `maxmark` before calling the function
- the function is "stand-alone", i.e. not connected to a variable in main (except temp…?)
- the code is more readable and less likely to have errors

Why do we use parameters?

- Parameters are used to send data to the function. For example, the `sqrt` function has to be told which number it must find the square root of.
- Parameters allow the same instructions in the function to be reused for different values. For example, in the program above, the function `readmark` can be used for a test where the maximum mark is 30 and it can also be used for a test where the maximum mark is 70.

Data Diagrams

It may be useful to use simple diagrams to illustrate how data moves between main and the function.

Memory layout for a function with no parameters



There are independent areas of memory for main's data (local variables) and for the function's data (local variables). No data moves directly between main and the function. This is the reason for the word `void` in the prototype and the empty parenthesis after the name. The function's area of memory is only used during the execution of the function (as the instructions of the function are being executed). However, both main and the function can access the ***global variables***, i.e. the variables that are declared at the top of the program.

Memory layout for a function with parameters



There are independent areas of memory for main's data (local variables) and for the function's data (local variables). Copies of the data can be passed directly from main to the function by using parameters. The values to be passed appear in the parenthesis when the function is called. In the function's world, the parameters act as variables along with any local variables. Both main and the function can access the ***global variables*** (i.e. the variables that are declared at the top of the program) but neither can access the either one's ***local variables***.

return and local variables

We have seen the use of parameters **passing** data from a calling function to another function. Now we consider data transferred in the opposite direction – from a function back to the calling function. This is known as **returning** data.

The function prototype must indicate whether or not there is a return, just as it indicates whether or not there are parameters. In the prototype, parameters are indicated in the parentheses and returns are indicated before the name. Note that `void` before the name means no data will be returned.

IMPORTANT: a function may have several parameters, but there is only ever ONE value returned.

Example: A function with parameters (arguments) that returns a value

```
#include <iostream>
using namespace std;

float addthree(float a, int b, int c);

int x;
float y;

int main() {
  x = 3;
  y = addthree(2.0, x, x * 6 % 7);
  cout << "The result is " << y;
}

float addthree(float a, int b, int c) {
  float result;
  result = a + b + c;
  return result;
}
```

Interesting things to notice in this program:

- the function name has `float` in front of it and not `void` because a real number is returned
- put `void` in front of a function that is not going to return any result
- the function ***returns*** a value to main which is loaded into variable `y`
- the call to the function is an assignment statement in order to store the returned value
- `y`, `result` and the start of the prototype are all the same data type (`float` in this example)
- if the prototype shows a return, the word `return` must appear somewhere in the definition.
- a function ends (and the calling function resumes) either when the processor reaches the function's closing brace or, when a return statement runs
- the function has a ***local variable*** that belongs only to the function

Local variables

Local variables live inside a function. They are used for storage inside the function. main (and other functions) do not know about the local variables that belong to other functions. They help to make "stand-alone" functions, i.e. there are no connections between the function and main. This makes functions very portable, i.e. a function can easily be copied into another program or the function can be stored in a library from where it can be used by other programs.

Global variables are declared at the top of the program. All our programs up to now have used global variables. Global variables can be used by main and also by any function in the program.

main is a function. It has a name (main) and it has a return data type (int) and it has parenthesis. main can also have local variables. Any variable that is used only for storage inside main (and not in any function) can be declared as a local variable in main and not as a global variable. This makes programs easier to understand as it is more obvious which variables are used where.

Example: Reading two test results – version 3 (two parameters and return)

Look back at the example program to read test results – version 2. We had to use a global variable to send data to the function. Let us write that program again using parameters.

```
#include <iostream>
using namespace std;

int readmark(int n, int maxmark);

int test1, test2, result;

/* note - test 1 is out of 30 and test 2 is out of 70 */

int main() {
  test1 = readmark(1, 30);
  test2 = readmark(2, 70);
  result = test1 + test2;
  cout << "Final result is " << result;
}

int readmark(int n, int maxmark) {
  int mark;
  cout << "Enter the mark for test " << n << ": ";
  cin >> mark;
  while ((mark < 0) || (mark > maxmark)) {
    cout << "ERROR: mark must be from 0 to " << maxmark << endl;
    cout << "Enter the mark again ";
    cin >> mark;
  }
  return mark;
}
```

This is the third time we have written this program. The first time we had no parameters and no return. The second time we used parameters but no return. This time we use parameters, return and local variables.

This version of the function is now completely **stand-alone**. It has no connection with main. Note that the variable `temp` has disappeared from this version of the program. The function does not make use of any global variables. The function is now portable – ready to be copied into another program (or put in a library).

This version of the program has three global variables which are:

```
int test1, test2, result;
```

We note that the function does not use any of these variables. We could therefore make these variables local variables in main.

How useful is `return`?

1. We can send a result back to main (or the calling function)
2. We can stop the function at any point

It is useful to remember that return does these two things (not just one of them). We can make use of this information in order to make functions simpler and easier to read. Here is an example:

Example (a): The absolute value function (*using return only to send back a result*)

Actually we don't need a function to calculate absolute value because we can use library functions to do this. But it makes an interesting example:

```
int absolute (int x) {
  int result;
  if (x > 0) {
    result = x;
  } else {
    result = -1 * x;
  }
  return result;
}
```

This is only the function definition. Write a `main` program that calls the function. Programmers often look only at functions without looking at the entire program.

Example (b): The absolute value function (*also using return to stop the program*)

Let us write the absolute function again and make it simpler and easier to read:

```
int absolute(int x) {
  if (x > 0) {
    return x;
  }
  return -1 * x;
}
```

Interesting things to notice in this function:

- the word `return` occurs more than once
- there is no local variable and there is no else

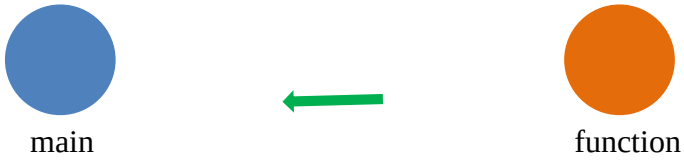
Void functions

A function that does not return a value is called a void function. Instead of a data type, such functions have the word `void` stated before the function name. We have already looked at several examples of void functions. Void functions do not need a return statement. However, you can include a return statement if you want to stop the function early. In this case, the statement would be `return;` as there is no value to return to main.

Data Diagrams

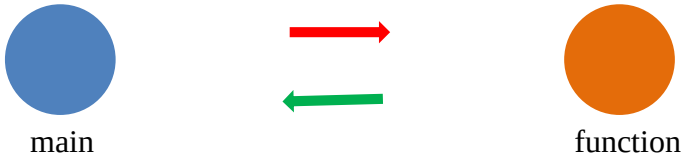
It may be useful to use simple diagrams to illustrate how data moves between main and the function.

Memory layout for a function with a return



There are independent areas of memory for main's data (local variables) and for the function's data (local variables). No data moves directly from main to the function (i.e. no parameters). The function's area of memory is only used during the execution of the function (as the instructions of the function are being executed). Data is returned from the function to main. The data is loaded into a variable in main by using an assignment statement. The data type must be stated before the name of the function. Both main and the function can access the ***global variables*** (i.e. the variables that are declared at the top of the program) but neither can access the either one's ***local variables***.

Memory layout for a function with parameters and return



There are independent areas of memory for main's data (local variables) and for the function's data (local variables). Copies of the data can be passed directly from main to the function by using parameters. The values to be passed appear in the parenthesis when the function is called. In the function's world, the parameters act as variables along with any local variables. The function's area of memory is only used during the execution of the function (as the instructions of the function are being executed). Data is returned from the function to main. The data is loaded into a variable in main by using an assignment statement. The data type must be stated before the name of the function. Both main and the function can access the ***global variables*** (i.e. the variables that are declared at the top of the program) but neither can access the either one's ***local variables***.

Array parameters

We can send an array to a function as a parameter. Array parameters do not have a fixed size. We use the empty square brackets to indicate that the parameter is an array but the size is not given. This means that the function can be used several times with different arrays of different sizes. It is important to include another parameter to inform the function how many elements of the array we are interested in.

Example: A function to calculate the average of numbers in an array

```
#include <iostream>
using namespace std;

float getaverage(float arr[], int count);
```

```
// there are no global variables

int main() {
  float results[100], average;
  int num, i;
  cout << "How many test results do you have? ";
  cin >> num;
  for (i = 0; i < num; i++) {
    cout << "Enter result " << i+1 << ": ";
    cin >> results[i];
  }
  average = getaverage(results, num);
  cout << "The class average is " << average << endl;
}

float getaverage(float arr[], int count) {
  int i;
  float total, result;
  total = 0.0;
  for (i = 0; i < count; i++) {
    total = total + arr[i];
  }
  result = total / count;
  return result;
}
```

Interesting things to notice in this program:

- the function has three local variables – main may not use these variables
- main has four local variables – the function may not use these varables
- the function has a local variable called `i` and main also has a local variable called `i`. It is important not to have a global variable called `i` which could cause confusion.
- the function is **stand-alone** (does not use any global variables). This means that it could be put in a library for other programmers to use.
- there are no global variables at all. This is good practice because it ensures that any functions are stand-alone. However, there are times when it is useful to use global variables.
- the function will work on an array of any size – `count` informs the function how many elements of the array we are actually interested in.
- what happens if I enter more than 100 test results?